

User's Guide

Display Stream Compression (DSC) 1.2 Encoder

V2.5.1, February 16, 2023

Trilinear

Cores against chaos.

TABLE OF CONTENTS

1	INTRODUCTION	3
1.1	STANDARDS COMPLIANCE	3
1.2	SUPPORTED FEATURE LIST	3
1.3	LIMITATIONS	3
1.4	UNSUPPORTED FEATURES	3
1.5	ASSUMPTIONS	4
1.6	REFERENCED DOCUMENTS	4
2	FUNCTIONAL DESCRIPTION	5
2.1	TOP LEVEL DESCRIPTION	5
2.1.1	Functional Block Diagram	5
2.1.2	Encode Command Processor	6
2.2	PARAMETERIZATION	7
2.3	CORE CLOCK DOMAINS	8
2.4	SYNCHRONOUS RESET	9
2.5	CLOCK DOMAIN CROSSING SYNCHRONIZATION	9
2.6	EMBEDDED MEMORIES	11
2.6.1	Embedded Memory BIST Support	11
3	INTERFACE DESCRIPTION	12
3.1	CORE IO TABLE	12
3.2	HOST PROCESSOR INTERFACE	13
3.2.1	APB Write Cycle	13
3.2.2	APB Read Cycle	14
3.2.3	Host Interrupts	14
3.3	AXI4 INTERFACE	15
3.3.1	AXI4 Handshake Signals	15
3.3.2	Input Streaming Interface	15
3.3.3	Input Data Format	16
3.3.4	Output Streaming Interface	17
3.3.5	Output Data Format	17
4	DETAILED OPERATIONAL DESCRIPTION	19
4.1	CORE INITIALIZATION	19
4.2	PICTURE PARAMETER SET SUPPORT	19
4.3	ENCODER OPERATION	20
4.3.1	Single Step Encode Operation	20
4.3.2	Free Run Encode Operation	20
4.3.3	Selecting the Slice Width and Height	21
4.3.4	Variable Bit Rate Mode	21
4.4	GENERAL PURPOSE TIMER	21
5	CONTROL AND STATUS REGISTERS	22
5.1	ENCODER CONTROL REGISTERS	22

1 Introduction

The Display Stream Compression (DSC) standard from the Video Electronics Standards Association (VESA) offers visually lossless compression for high definition (HD) video in broadcast, automotive, medical and consumer electronics applications. The Trilinear Technologies DSC 1.2a Encoder offers real time compression of HD streams with resolutions from 480p up to 8K. The core is fully configurable with a picture parameter set specified for each frame and able to be reprogrammed on the fly.

1.1 Standards Compliance

The DSC Encoder core described by this document is fully compliant with the VESA Display Stream Compression standard. The core design implements an AMBA APB-4 interface and an AXI-Stream interface both of which are fully compliant with applicable specifications from ARM.

1.2 Supported Feature List

DSC 1.2 Encoder Functionality

- Display Stream Compression Support
 - Compliance tested core
 - Optional PPS parsing
 - MPP, BP and MMAP support
 - Arbitrary slice size support
 - 4:4:4 and 4:2:2 sampling support
- DSC 1.2 Support
 - Deep color support, 8, 10, 12, 14 and 16bpc
 - Simple and native 4:2:2 and 4:2:0 modes
- AMBA Interfaces
 - APB-4 host interface, 32-bit with byte write
 - AXI-Stream input interface
 - AXI-Stream output interface
- High Performance Design
 - 4K video at 60Hz
 - 8K video at 30Hz (ASIC only)
- Robust Software Support
 - Free run or single step encoding
 - Interrupt driven operation
 - Low CPU overhead

1.3 Limitations

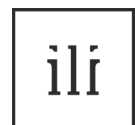
The DSC Encoder core design described by this document may implement specific features in a limited way. These features are listed here.

- | | |
|------|--|
| DL-1 | The DSC Encoder core requires intervention from the host processor in when an error condition is detected. |
|------|--|

1.4 Unsupported Features

Some of the features of the Display Stream Compression specification are currently not supported.

- | | |
|------|---|
| UF-1 | The Native 4:2:0 format is currently not supported. |
|------|---|



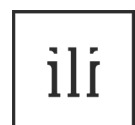
1.5 Assumptions

Specific assumptions have been made in the design and development of the DSC encoder core which are listed here.

- DA-1 The input and output interfaces have been designed to support the AXI-Stream interface. The core does not implement any rate control buffering. Any required rate control logic is left to the system implementation.

1.6 Referenced Documents

- (1) VESA, "VESA Display Stream Compression (DSC) Standard", v1.2a, January 18, 2017
- (2) ARM, "AMBA® APB Protocol, Version: 2.0", Issue C, April 13, 2010
- (3) ARM, "AMBA 4 AXI4-Stream Protocol", Issue A, March 3, 2010
- (4) ARM, "AMBA AXI and ACE Protocol Specification", Issue E, February 22, 2013



2 Functional Description

The Trilinear Display Stream Compression Encoder core implements a VESA compliant DSC 1.2 Digital Video Encoder capable of processing input data streams at rates up to 1600 MPixels per second. The functional design is partitioned into multiple blocks which together comprise the video data processing pipeline. An input image may be partitioned into multiple vertical slices which are then processed in parallel to increase the overall throughput of the design. A central Sequence Controller block governs the pixel processing of the video encoder pipeline. This function forms the primary encode controller and can be operated under host processor control.

2.1 Top Level Description

The Trilinear DSC encoder core is designed to efficiently process input frames as a series of vertical slices which are compressed using a combination of value prediction and color history techniques. The DSC standard attempts to achieve near lossless compression through intra-frame coding techniques. The Trilinear encoder implementation supports the full range of prediction tools available in the DSC 1.2 specification including the Block Prediction.

2.1.1 Functional Block Diagram

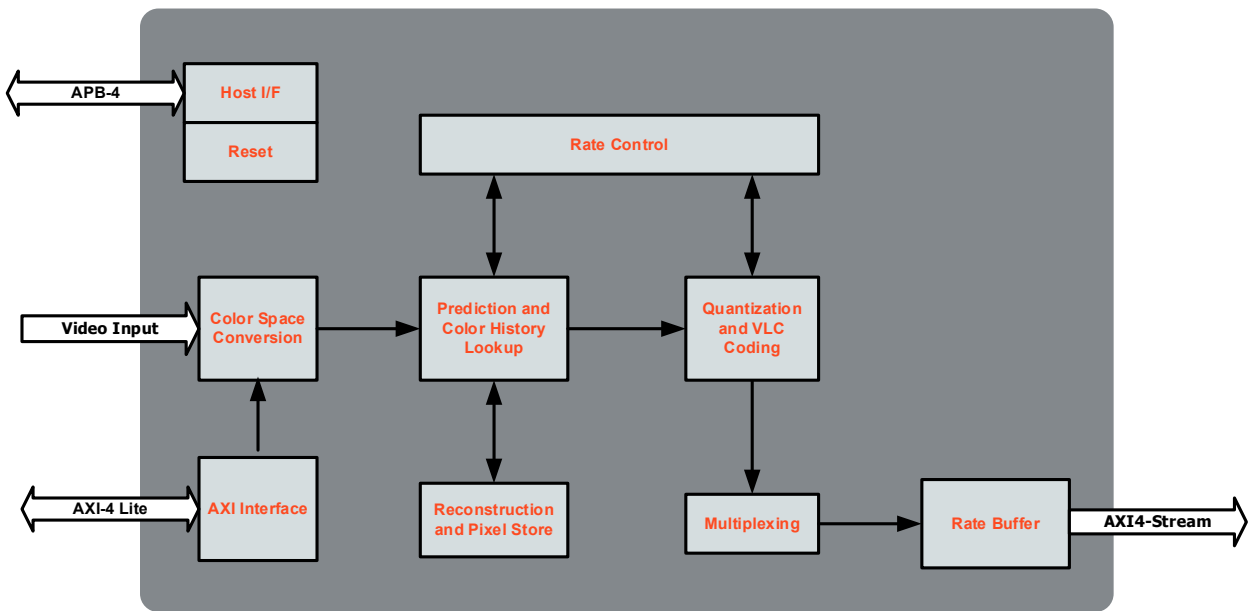


Table 2-1 : Top Level Block Diagram



2.1.2 Encode Command Processor

The encoder operations are controlled through a central command block that accepts input from the host processor interface to begin and end the encode process. In a real-time system, this command structure can be used to effectively manage the data stream with as much or as little host processor intervention needed to meet the system requirements. The command block may also be used to control the encode process on a slice by slice or a frame by frame basis. The command block accepts one of the following commands:

Command Number	Name	Description
0	Encoder Stop	Places the encoder in the stopped state. This is the default state of the encoder. Issuing this command while already stopped will not affect the internal state of the encoder. Writing this command while the encoder is active will cause the encoder to stop after the completion of the current command.
1	Encoder Reset	Used to reset the internal state of the encoder. The states of the internal encoding machine will be reset to the same values used at a power on reset. The values of the host register set will be unaffected by this command.
2	Encode Chunk	Instructs the encoder to process a single chunk of data. When multiple slice processors are implemented, this command will cause each slice processor to encode a single output chunk.
3	Encode Slice	Instructs the encoder to process a single vertical slice of data. When multiple slice processors are implemented, this command will cause each slice processor to encode a single slice. The amount of an input frame that is processed by this command will depend on the programmed slice height, number of slices per frame and the number of slice processors implemented.
4	Encode Frame	Instructs the encoder to process an entire frame of data using the picture parameter set from the host interface registers.
5	Encode Free Run	Places the encoder in the free run mode to allow for image frames to be accepted directly from the AXI-S input interface.

Table 2-2 : Control Commands

The ENCODER_ACTIVE register will be set to a 1 while the sequencer is encoding pixels. The commands in the table above may be written to the ENCODE_COMMAND register at any time the ENCODER_ACTIVE register is set to a 0. Commands that are issued while the ENCODER_ACTIVE register is set will be ignored until the current command has completed with the exception of the Encoder Reset command which will immediately halt all processing by the encoder core and return the internal compression engine back to the default state.

2.2 Parameterization

The HDL source code includes a set of parameters which are useful in controlling the implementation of the core per the system requirements. These values may be set individually as needed. Parameters which are not explicitly set will take the default value as specified in the table below.

Parameter	Default Value	Description
pSLICE_PROCESSOR_COUNT	4	{1, 2, 4, 8, 16} Number of hardware slice processors implemented to encode the maximum input pixel rate. This number must be a power of 2. Values over 16 are not recommended.
pMAX_LINE_SIZE	4096	{1024, 2048, 4096, 8192} Maximum number of pixels per input line to be encoded. This parameter controls the size of the internal line buffer used to hold the line to be encoded.
pINCLUDE_BLOCK_PREDICTION	1	{1, 0} When set to 1, the implementation of the core will include the block prediction function.
pDEBUG_MESSAGES	0	{1, 0} This parameter is provided for simulation purposes only and instructs the core design to display detailed operational information about the core as input data is begin encoded.

Table 2-3 : Core Parameters



2.3 Core Clock Domains

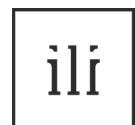
The DSC Encoder core implementation utilizes three clock domains for the primary core functions. The AMBA APB-4 slave interface operates using the APB_CLK. The slave interface is used to provide configuration of the encoding process and status information for management by the host processor. The uncompressed video input stream is provided via the streaming input port and is clocked by the AXI_CLK input. The output of the encoded bit stream is provided using an AXI-Stream interface. The output interface is clocked by the same AXI_CLK as the input interface.

Function	Clock Domain	Min
Host	APB_CLK	33MHz
Engine	DSC_CLK	33MHz*
Input	AXI_CLK	10MHz*
Output	AXI_CLK	10MHz*

Table 2-4 : Core Clock Frequencies

* The encoded video mode determines the minimum clock rates. Please refer to the integration guide for sample data rates.

Each clock domain within the core design is intended to operate independently of all other clock domains. However, in some applications, the APB and AXI domains may be tied together without any adverse effects.



2.4 Synchronous Reset

The encoder core reset is provided using a centralized synchronization block which accepts the external asynchronous reset or a host generated soft reset. On the detection of one or more of these reset sources, the internal logic creates an independent, synchronous reset for each internal clock domain. In the case of the soft reset, the host clock domain, APB_CLK, is not reset. This domain includes the host control and configuration registers which are not modified due to the assertion of the soft reset.

Each synchronous reset is generated from an internal multi-stage shift register. The asynchronous input signal sets the shift register to output a 0. After the specified number of clock cycles, the reset will be synchronously cleared. The value of the reset length for the core is fixed at eight (8) clock cycles.

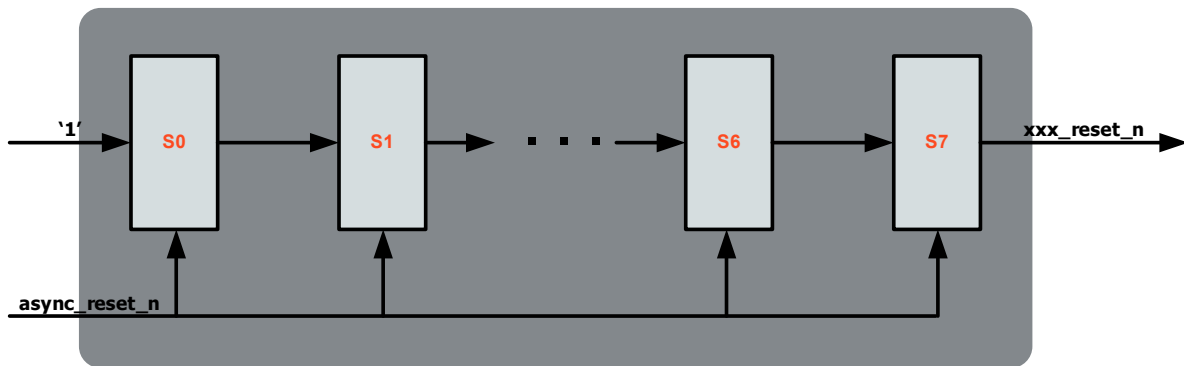


Figure 2-5 : Generic Reset Circuit

During the operation of system level testing such as scan testing, the reset circuit must be bypassed for the tester to have direct control of the reset for each internal flop. This control is achieved by setting the top level pin `async_test_mode` to a 1 during system level testing.

2.5 Clock Domain Crossing Synchronization

Each IP core from Trilinear Technologies includes an abstract implementation of a metastable resistant register primitive. These registers are included in the `gprim_sync_stage.sv` and `gprim_sync2_stage.sv` design files. All individual signals that are required to transition from one clock domain to another are synchronized using these registers.

The use of a generic primitive module allows for the rapid replacement of the primitives with an application specific primitive from the cell library of the target technology. Please refer to the documentation for the process specific cell library for more information.

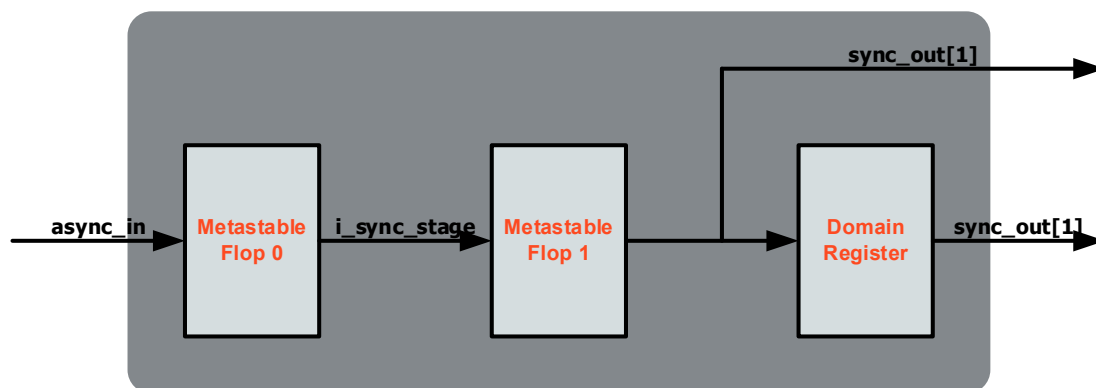


Figure 2-6 : Metastable Register Implementation (2-stage)

2.6 Embedded Memories

The following tables summarize the SRAM elements which are implemented in the design. The size of the Line Buffer memory depends on the setting of the parameter `pMAX_LINE_SIZE` and affects the maximum horizontal input resolution supported by the core.

The following memories are instantiated only once in the core design. These are the top-level command and control buffers.

Functional Block	Count	Depth	Width	Total Bits	Ports
PPS	2	256	8	4K	1r / 1w

Table 2-7 : Embedded Memories

The following memories are implemented within each slice processor. The hardware slice processor count parameter will determine the total number of instances.

Functional Block	Count	Depth	Width	Total Bits	Ports
Slice Buffer (1)	1	2048 / SPC	192	6K	1r / 1w
Input Buffer (2)	1	32	192	6K	1r / 1w
Line Buffer (3)	1	4096	96	384K	1r / 1w
Output Buffer	1	256	192	48K	1r / 1w

Table 2-8 : Memories Per Slice Processor

- (1) The slice buffer depth depends on the parameter `pMAX_LINE_SIZE` and `pSLICE_PROCESSOR_COUNT`. The equation for the size of this memory is

$$\text{depth} = \text{pMAX_LINE_SIZE} * 2 / (\text{pSLICE_PROCESSOR_COUNT} * 4)$$

- (2) For designs implementing less than 4 pixels per cycle on the input port or less than 16 bits per color plane may be able to optimize this memory. Please contact Trilinear technical support for more information.
- (3) The depth of this memory is dependent on the value of the parameter `pMAX_LINE_SIZE`.

2.6.1 Embedded Memory BIST Support

Each of the internal memories is abstracted through a generic memory layer that includes a single input vector and a single output vector for the connection of BIST signaling. The BIST input and output vector are routed to the top level of the core for connection to system level test logic. The implementation of the internal core design does not depend on any specific order or function of these signals.



Figure 2-9 : BIST I/O Support

The generic memory models included with the core do not implement any BIST functionality. The BIST input vector is ignored and the BIST output vector is tied to a zero.

3 Interface Description

The following sections describe each interface in detail and provide reference timing diagrams to aide in the understanding of the relationships between the signals.

3.1 Core IO Table

The following table details the input and output ports of the DSC Encoder core.

Port	Dir	Domain	Description
APB Host Interface			
apb_clk	in	APB	Host processor bus interface clock.
apb_select	in	APB	Transmitter core select. Active high to accept host bus transactions.
apb_enable	in	APB	Data transfer enable signal for the APB interface
apb_write	in	APB	Determines write (1) or read ('0') transactions
apb_strobe [3:0]	in	APB	Byte lane write strobes.
apb_address [11:0]	in	APB	Host address, must be DWORD aligned.
apb_protect [2:0]	in	APB	APB access control (currently unused).
apb_wdata [31:0]	in	APB	Data to be written when apb_write = 1 during the enable cycle.
apb_rdata [31:0]	out	APB	Read data returned during the enable cycle.
apb_ready	out	APB	Ready flag for wait state insertion.
apb_slave_error	out	APB	Asserted to indicate a slave transfer error.
apb_int	out	APB	Active high, level based interrupt.
Misc Control Inputs			
reset	in	ASYNC	Global core reset.
test_mode	in	ASYNC	Enables scan test mode logic.
Encoder Compression Engine Interface			
dsc_clk	in	DSC	Encoding engine core clock
AXI Stream Input Interface			
axi_clk	in	AXI	Input domain AXI stream clock.
axi_tvalid_in	in	AXI	Input data valid flag.
axi_tready_in	out	AXI	Encoder core input ready flag.
axi_line_in	in	AXI	End of line indicator, active high.
axi_frame_in	in	AXI	End of frame indicator, active high.
axi_tdata_in [192:0]	in	AXI	Supports up to 4 pixels per cycle at 16 bits per color.
AXI Stream Output Interface			
axi_tvalid_out	out	AXI	Encoder output data valid flag.
axi_tready_out	in	AXI	System transfer ready indicator.
axi_line_out	out	AXI	End of line indicator, active high.
axi_frame_out	out	AXI	End of frame indicator, active high.
axi_tdata_out [192:0]	out	AXI	Compressed stream data output.

Table 3-1 : Core IO Table

3.2 Host Processor Interface

The host processor interface is implemented using the Advanced Peripheral Bus (APB) interface as described by the AMBA APB-4 Specification. This interface is optimized for reduced power and reduced interface complexity. Utilizing this standard as the primary interface has the dual advantages of being low complexity and providing an easy migration path for interface translation from many other bus standards.

3.2.1 APB Write Cycle

The AMBA APB write transfer begins with the address, write signal, and write data set to their proper values on the first rising edge of the clock. The first clock cycle of the transfer is called the SETUP cycle. On the second rising edge of the clock, the enable signal is asserted and the ENABLE cycle is entered. The address, data, and control signals all remain valid through both cycles of the transfer. The transfer completes on the following rising edge of the clock. Both the SETUP and ENABLE cycles are completed only when the READY signal is high. When accessing specific elements of the core, the READY signal may be driven low to extend the current cycle to allow the core time to complete the requested write operation.

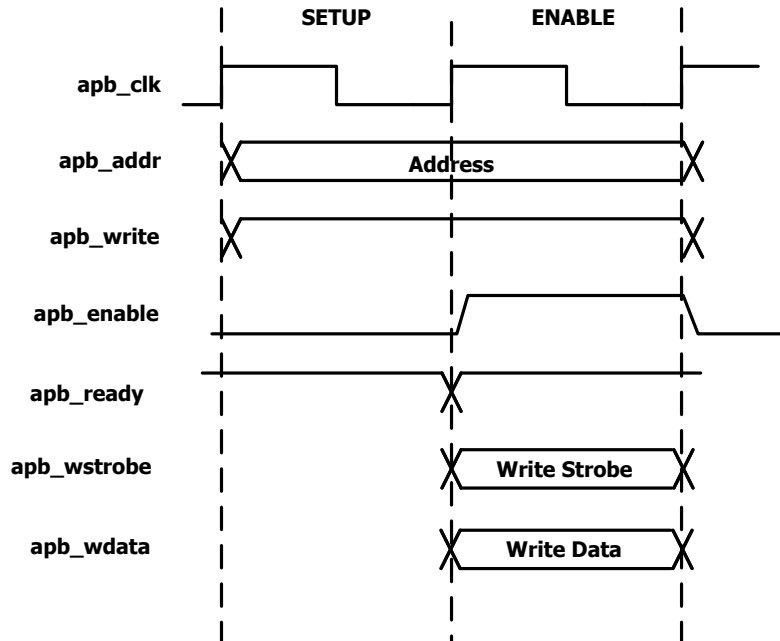


Figure 3-1 : AMBA APB Write Cycle

The APB-4 write protocol includes the ability to perform byte lane writes using four strobe signal qualifiers. The *apb_strobe* input contains one-byte lane write enable signal for each of the four bytes in the 32-bit DWORD interface. The strobes are mapped in little endian format,

<i>apb_strobe</i>	<i>apb_wdata</i>
0	7:0

1	15:8
2	23:16
3	31:24

Table 3-2 : Write Strobe to Byte Lane Mapping

3.2.2 APB Read Cycle

The AMBA APB read transfer begins with the SETUP cycle on the first rising edge of the clock with the address and control signals at their proper values. As with the write transfer, the enable signal is asserted on the next rising edge marking the beginning of the ENABLE cycle. The slave peripheral must provide data during this cycle. The read data is sampled on the next rising edge of the clock at the end of the ENABLE cycle.

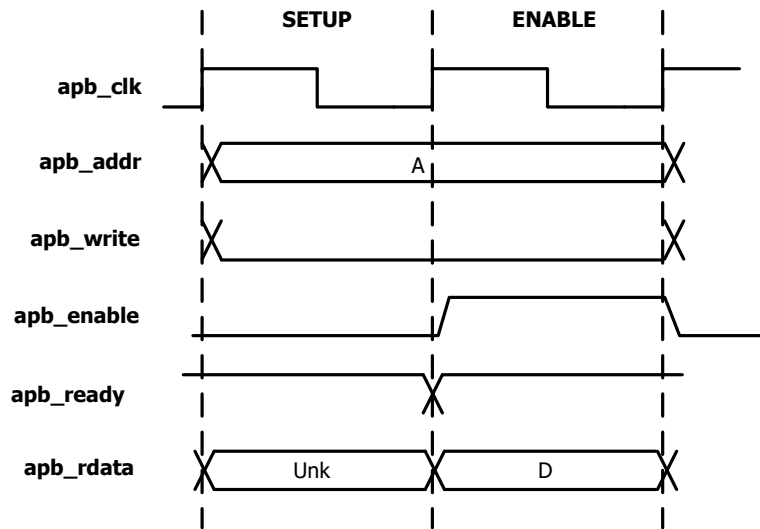


Figure 3-2 : AMBA APB Read Cycle

3.2.3 Host Interrupts

The encoder core generates an interrupt to the host processor under several selectable operating conditions. When one or more of these conditions is detected, the encoder core sets the `apb_int` output signal high. All interrupts are cleared when a read is performed from the `INTERRUPT_PENDING` register.

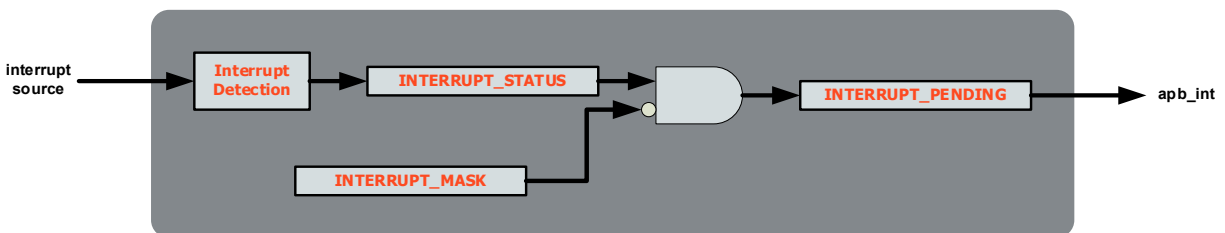


Figure 3-3 : Interrupt Detection Logic

Upon detection of an interrupt, the encoder core will immediately update the status register `INTERRUPT_STATUS`. The bits of this register are set independently of the state of the individual interrupt mask bits. The purpose of the status register is to detect events within the core without asserting an interrupt. These bits may be used in an interrupt polling system.

Each bit in the `INTERRUPT_STATUS` register is masked with the corresponding bit in the `INTERRUPT_MASK` register. Any interrupts which are not masks and asserted are transferred to the `INTERRUPT_PENDING` register. The state of this register determines the assertion of the `apb_int` signal. When the interrupt pending register is non-zero, the external interrupt signal will be asserted high and will remain high until the host processor reads from the `INTERRUPT_PENDING` register. Reading from this register will clear all interrupt sources.

3.3 AXI4 Interface

3.3.1 AXI4 Handshake Signals

Data transfer over the AMBA AXI4-Stream bus is completed using a two signal handshake process. The `axi_tvalid` and `axi_tready` signals determine when data is passed across the interface. For a valid transfer to occur, both the `axi_tready` and `axi_tvalid` signals must be asserted during the same `axi_clk` cycle.

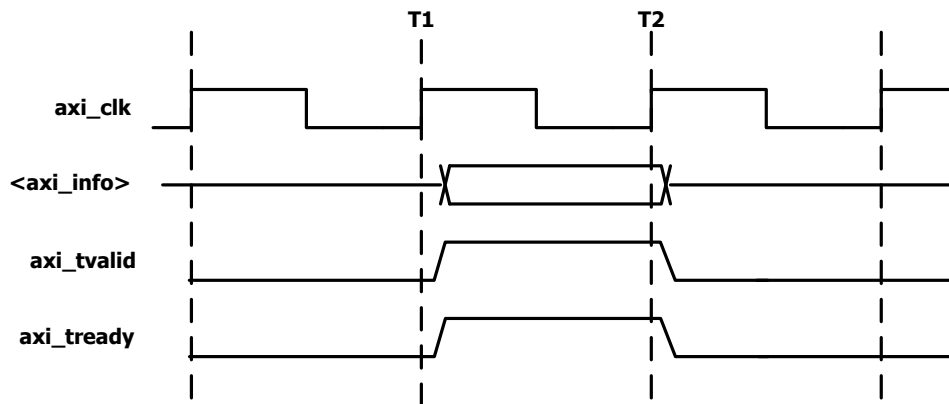


Table 3-4 : AXI Channel Handshake Signals

3.3.2 Input Streaming Interface

In addition to the handshake signals required for the transfer of data over the AXI4-S input port, the encoder core defines two additional control flags used for the framing of image data. The single bit control lines, `axi_tline_in` and `axi_tframe_in`, frame the active data into complete lines and frames. The polarity of the framing and control bits is active high.

Vertical timing is framed using the `axi_tframe_in` signal which indicates the end of frame N-1 and the beginning of frame N. The vertical framing signal is active high and must be at least one (1) `axi_clk` cycle wide. The rising edge of this signal causes the encoder to advance the current frame. Holding this signal high for longer than one (1) `axi_clk` cycle will not cause any adverse effects.

At the rising edge of each input vertical framing pulse, the input image data interface will reset key elements of the data path. This automatic reset provides for a robust user interface that is able to recover from many possible types interface errors in one vertical interval or less.

Similarly, the horizontal framing indicator, *axi_tline_in*, marks the end of line N-1 and the beginning of line N. The input signal is active high and must be asserted for a minimum of one (1) *axi_clk* cycle. During the vertical blanking interval, the *axi_tline_in* signal may be asserted or omitted without changing the behavior of the encoder core.

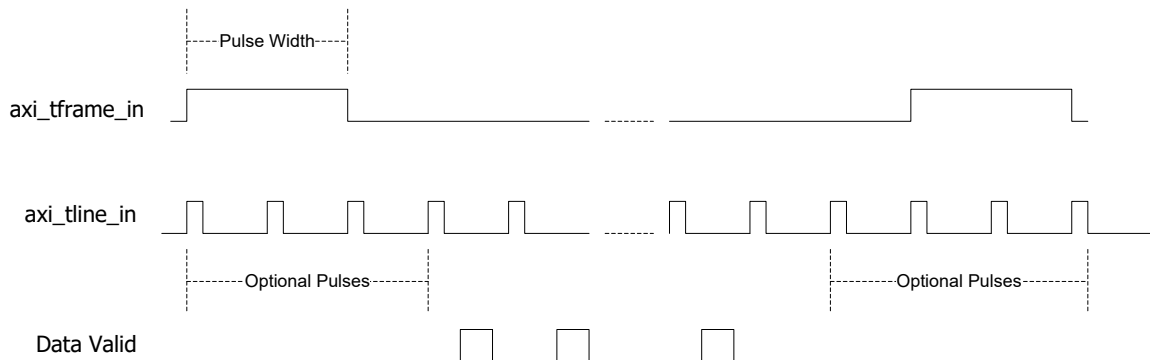


Figure 3-3 : User Interface Vertical Timing

3.3.3 Input Data Format

The Display Stream Compression core supports input bit depths of 8, 10, 12, 14 and 16 bits per color using three separate color channels. The complete video data signal interface is fixed at 48-bits per pixel and supports the capture of 1, 2 or 4 pixels per clock cycle. Each 48-bit video data input is organized into color channels as shown below for the 4:4:4 color format in either the RGB or YCbCr color space.

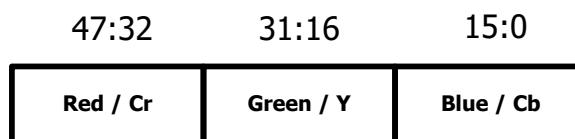


Figure 3-4 : Input Pixel Layout

For the support of the 4:2:2 and 4:2:0 color formats, only the most significant bits of the input data capture port are used for valid pixels.

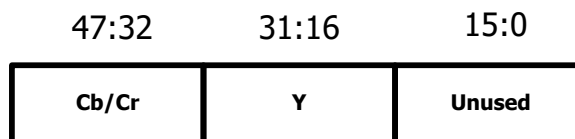


Figure 3-5 : 4:2:2 and 4:2:0 Input Pixel Layout

Within each 16-bit color channel, a system implementation may use less than the full 16-bit color depth per channel. In these cases, only the most significant bits of the color channel need to be connected to valid data. The unused, least significant bits should be tied to zero.

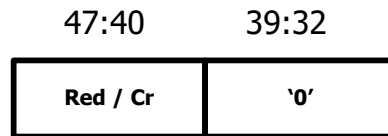


Figure 3-6 : Color Data Organization, 8-bit Example

The above example illustrates the use of eight bits per color channel. For the purposes of clarity, only the Red channel is shown. The Green and Blue channels would be similarly mapped. In this case, the eight MSBs (47:40) are connected to the eight bits of the input data (7:0). The unused bits (39:32) would be tied low.

The DSC encoder is capable of accepting multiple pixels per cycle. The pixel input mapping is replicated four (4) times to provide distinct input pins for up to four pixels. For the image data inputs, a wide array, [191:0], is used to provide pixels to the source interface.

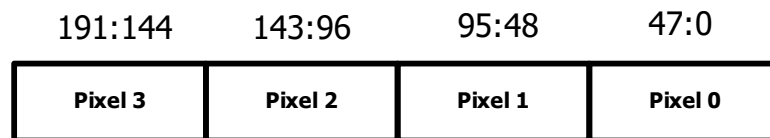


Figure 3-7 : Source Video Data Mapping

Pixel 0 represents the left-most pixel of the set while Pixel 3 represents the right-most pixel in the set. When the input is configured to be either 2 or 4 pixels, the resolution of the source image must be a multiple of either 2 or 4 pixels respectively. The input signals for unused pixel inputs should be tied to zero.

3.3.4 Output Streaming Interface

The compressed image data is output using an AXI4-Stream interface. The user defined framing signals, *axi_tframe_out* and *axi_tline_out* are used to frame the compressed output data. In addition to the image data, the core can optionally output the 128-byte picture parameter set at the top of each output frame. When the XXX_ENABLE bit is set to a 1 in the host register interface, the first 128 bytes after the assertion of *axi_tframe_out* are a part of the PPS.

3.3.5 Output Data Format

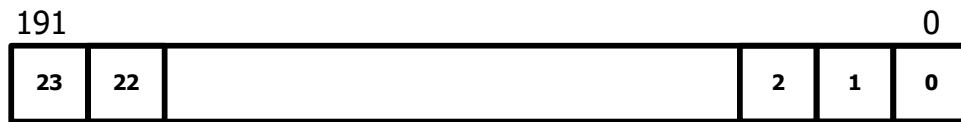
Compressed output frames are transferred through the *axi_tdata_out* port using a programmable bus width. The configuration of the output data width is controlled using the AXI_OUTPUT_MODE register.

AXI_OUTPUT_MODE [2:0]	DATA_SIZE
000	8-bit
001	16-bit
010	24-bit

011	32-bit
100	48-bit
101	64-bit
110	96-bit
111	192-bit

Table 3-5 : AXI_OUTPUT_MODE Settings

When the *axi_tvalid_out* and *axi_tready_out* are both active, a complete word is transferred from the internal staging FIFO. The data stream is big endian with byte 0 of the output port representing the first byte of the stream as shown above.

**Figure 3-8 : Output Byte Mapping**

If the transmission of the Picture Parameter Set before each encoded frame is enabled, the bytes of the PPS are output in order from 0 to 127. In this mode, the first byte of the compressed image is output as byte number 128. For example, the first compressed image byte would be byte 0 of output word 6 n in the 192-bit data size case. Each subsequent data word after the PPS represents the next compressed image bytes of the stream.

		Byte Number							
Cycle		7	6	5	4	3	2	1	0
0		Slice N							
1		Padded with 0x00				Slice N			
2		Slice N + 1							

Figure 3-9 : Output Slice Alignment

When multiple slices are implemented in the compressed data stream, any unused bytes in the last data word of the slice will be padded with zeros to align the entire compressed frame to the output bus boundary. The first byte of the next slice will begin at byte 0 of the next word.

4 Detailed Operational Description

The following sections serve as a reference for performing basic encoder operations with the Trilinear DSC Encoder core.

4.1 Core Initialization

Upon initial power up, the following sequence may be performed to return the core to a default state. This sequence may be used at any time to set the encoder core back to a known state.

1. write the reset command to the ENCODER_COMMAND register
2. wait for the ENCODER_ACTIVE bit to be cleared to zero
3. set the ENCODER_COMMAND register to the off state

The sequence will reset the stream buffer for processing and set the internal DSC compression engine back to the idle state. The core is then ready to begin the encoding process.

4.2 Picture Parameter Set Support

The transmission and application of the Picture Parameter Set is not included in the Display Stream Compression standard. The encoder implements the picture parameter set as a set of internal control registers. To prevent the host system from modifying the active PPS while a frame encode operation is in progress, the core includes a set of shadow registers that can be safely programmed before being committed to the active encode process.

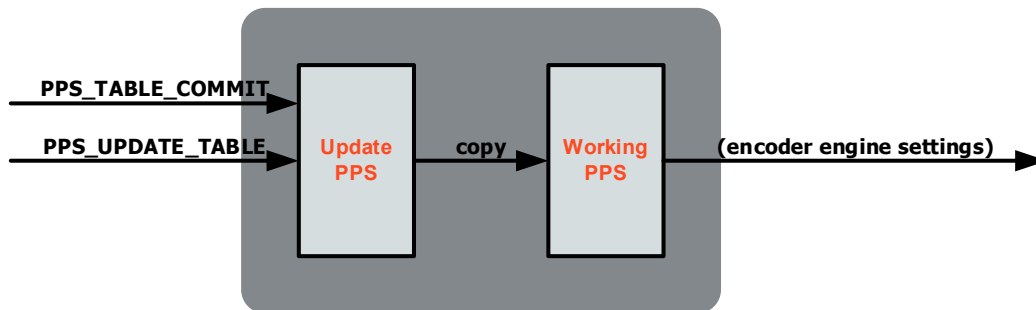


Figure 4-1 : Picture Parameter Set Update

The values for the Picture Parameter Set are first loaded into the shadow register set using the PPS_UPDATE_TABLE address locations. The PPS table may be randomly accessed and the settings will have no effect on the operation of the core until the PPS_TABLE_COMMIT register is set. Once the PPS_TABLE_COMMIT register is set and the compression engine is either idle or reaches the end of the current frame, the updated parameter set will be copied into the working parameter set which will be used for encoding the next frame. After writing to the PPS_TABLE_COMMIT address, the register will read back as a 1 until the compression engine completes the copy of the settings from the update PPS to the working PPS.

4.3 Encoder Operation

4.3.1 Single Step Encode Operation

After the proper initialization of the core, the host processor initializes the encode process by instructing the core to encode a complete frame or some portion of a frame. From the working picture parameter set (PPS), the core collects the information required to configure the memory interface, if needed, and the compression engine.

The process continues with the main picture compression loop which consists of a command and monitor phase. In the command phase, the host processor issues a frame or slice encode command which instructs the compression engine to encode the image data until a frame or slice boundary has been reached. Once the selected boundary has been reached, the core will generate an interrupt.

Upon receipt of the interrupt, the host processor reads the register set for status information. If any errors occurred during the compression process, the host processor implements any required error recovery algorithm. Once error recovery is complete or if no errors were detected in the compression process, the host processor issues the next frame or slice encode command.

In the single step encoding mode, the timing of all encode operations may be finely controlled while maintaining a minimum amount of processor intervention in the process. The compression complete condition may be determined either through an interrupt driven call back or by polling the ENCODER_STATUS register.

4.3.2 Free Run Encode Operation

When the encoder is set to the free run compression mode, the DSC core will compress frames until a new command is received.

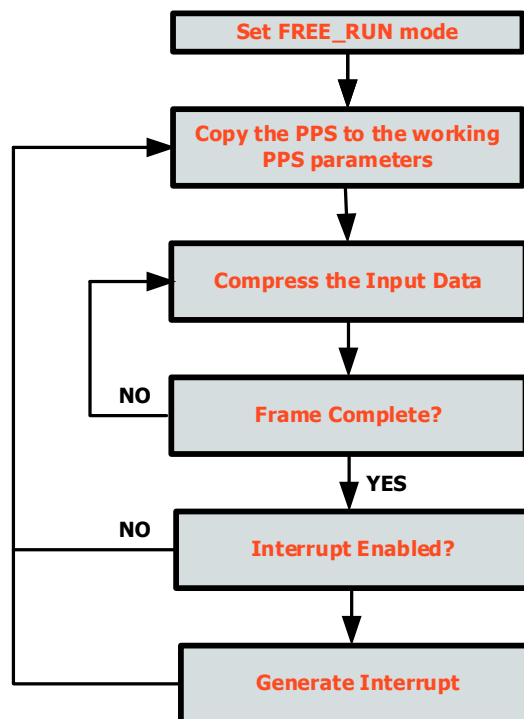


Table 5-4-2 : Free Run Flow

4.3.3 Selecting the Slice Width and Height

The selection of the height and width for each encoded slice is application specific and may depend on several factors including the rate of the DSC_CLK input. However, there are some limitations to specific operating modes of the DSC Encoder core that can affect the selection of these parameters. The equation for determining the number of slices per line is below.

$$\text{Slices Per Line} = 4/3 * (F_{\text{PIXEL_IN}} / F_{\text{DSC_CLK}})$$

Where:

$F_{\text{PIXEL_IN}}$ is the input pixel rate. $F_{\text{PIXEL_IN}} = F_{\text{CLK_IN}} * \text{number of pixels per cycle}$

$F_{\text{DSC_CLK}}$ is the frequency of the DSC domain processing clock

4.3.4 Variable Bit Rate Mode

The variable bit rate (VBR) mode of the Display Stream Compression protocol is enabled through the picture parameter set. Enabling this mode eliminates the zero padding at the end of a slice to maintain output muxword alignment. Please refer to the DSC specification for more details.

4.4 General Purpose Timer

The DSC Encoder core design includes a single general purpose 20-bit timer for use by the host processor system. The timer has a resolution of one microsecond and can provide a timing reference between 1 usec and 1 second. The timer may be configured to automatically reload the count value and may also generate an interrupt.

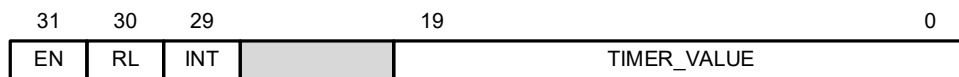


Figure 4-1 : Timer Control Bits

The timer is located at the offset GP_HOST_TIMER and contains four fields that are used to control the operation of the timer.

EN – Timer enable. This bit must be set to 1 for the timer to operate properly. When set to 0, the TIMER_VALUE field will always read back as the programmed reload value.

RL – Reload. The firmware should set this bit to a value of 1 for the timer to auto-reload once the terminal count is reached. If set to 0, the timer will count down to 0 and remain at zero until disabled and reenabled using the EN bit.

INT – Interrupt. When set to a 1, the timer will generate an interrupt when the count reaches zero. If the RELOAD bit is set to a 1, then the timer will generate an interrupt each time the count reaches zero.

TIMER_VALUE – Reload value / current value. On a write, this field contains the value to be loaded into the timer when the timer is either disabled or when the timer reaches zero and the reload bit is set to a 1.

5 Control and Status Registers

5.1 Encoder Control Registers

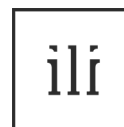
The DSC Encoder control register space contains all the information about the configuration of the input stream and the operation of the encoding engine. These registers are mapped as aligned 32-bit values on the APB host bus.

Any bits not specified in the table are to be considered reserved and will return '0' upon read.

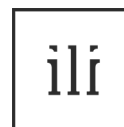
Offset	R/W	Definition
Encoder Configuration Field		
0x000	RW	ENCODER_COMMAND The host processor writes to this register to issues a command to the DSC compression engine. While a command is running, issuing the Encoder Off command will exit the current command at the next frame boundary. The Reset All command should only be used when the core has entered an error state. 3:0 – Parsing command 0000 = Encoder Stop - Disable encoding 0001 = Encoder Reset – Reset the encoder core 0010 = Encode Chunk– Encodes a single chunk from each slice processor 0011 = Encode Slice– Encodes a single slice from each slice processor 0100 = Encode Frame – Process through and encode the current frame 0101 = Free Run – Encode all frames as provided on the AXI4-S input interface
0x004	RO	ENCODER_ACTIVE Returns the current state of the compression engine. This bit will be set when the encoder is currently executing a previously issued command. 0 – This bit will read back as 1 while the encoder is processing a command including the free run command. While the compression engine is in the free run mode, this bit will always read back as 1. The bit will return to 0 when the command has been completed.
0x008	RW	PIXELS_PER_CYCLE Specifies the number of pixels to accept per clock cycle from the AXI input port. 2:0 – Supported values are 1, 2 and 4. The register defaults to 4 pixels per cycle after a core reset.
0x00C	RO	RESERVED
0x010	RW	ENCODER_TIMEOUT_COUNT Determines the number of inactive cycles before the encoder determines that processing the current stream has halted due to an error. When a timeout occurs, writing this register to a 0 will clear the timeout flag. 7:0 – The value of the timeout count is in terms of AXI_CLK periods and is calculated by multiplying the value of this register by 1024.
0x014	RO	RESERVED
0x018	RO	RESERVED
0x020	RO	ENCODED_FRAME_COUNT Reading this register returns the value of the active encoded frame count. The counter resets when a value of zero is written to this register. The value of the counter increases by one at the end of every encoded output frame.

		7:0 – Stores the frame count from 0 to 255. The counter will roll over to 0 automatically to provide a continuous count.
0x024	RO	FORCE_ENABLE After receiving an encode command, the encoder will wait for the detection of an input vertical sync before enabling the encoding engine. This control bit is used to override the logic that waits for the vertical sync and forces the engine to start the encoding process immediately. This control register should not be used in normal operation. 0 – Set to a 1 to override the built-in wait for vertical sync logic.
0x028	RW	RESERVED This register is used for debug purposes only. Writing any value other than zero to this control register will result in unpredictable encoder operation.
0x02C	RW	USEC_CLOCK_DIVIDER Determines the number of APB clock cycles per microsecond. This value is used to create an internal timer that is used for encoder timeout detection. The divided clock is also used for the operation of the internal timer. 9:0 - This value should be set to the nearest integer frequency, in MHz, of the APB clock.
0x030	RW	AXI_OUTPUT_MODE Sets the output mode (transfer bus width) of the encoder AXI bus. The number of bits should not be modified while the encoder is enabled. The encoder should be disabled prior to any modification of this register. 2:0 – Encoded data output mode select. 000: 8-bit 001: 16-bit 010: 24-bit 011: 32-bit 100: 48-bit 101: 64-bit 110: 96-bit 111: 192-bit
0x034	RW	GP_HOST_TIMER General purpose 24-bit timer for use by the host processor. The DSC encoder core does not use this timer for any internal function. The resolution of the timer is 1 usec as set by the USEC_CLOCK_DIVIDER register. 31 – TIMER_ENABLE: Set to a 1 to enable the timer. Writing a 0 to this bit will immediately disable the timer and no further events will be triggered. 30 – TIMER_RELOAD: Set to a 1 to automatically reload when the timer reaches zero. When set to 0, the timer will only run once. 29 – TIMER_INTERRUPT: Set to a 1 to generate an interrupt when the timer reaches zero. 23:0 – TIMER_VALUE: A write will set the reload value for the timer. A read will return the current value of the timer.
Slice Control Registers		
0x040	RW	SLICE_WIDTH_ALIGNED Informs the encoding engine that the slice boundary is aligned to the width of the picture. This bit should be set to a 1 when $SLICE_COUNT * SLICE_WIDTH = PIC_WIDTH$. Otherwise, this control bit should be set to a zero. 0 – Set by the encoding software to inform the core of the slice to picture alignment.
0x044	RW	WORDS_PER_SLICE

		The DSC encoder core outputs variable sized words on the AXI output interface qualified by the AXI control signals. This register informs the core how many output words are sent per slice. This value is used to multiplex the output slice after partitioning by the hardware slice processors. 11:0 – The value of this register is in fixed point format with 10 integer bits and 2 fractional bits. The integer portion is stored in the most significant bits.
0x048	RW	SLICES_PER_PROCESSOR Each hardware slice processor may be configured to encode multiple image slices per line. This register configures the number of DSC slices to be encoded by each internal hardware slice processor. WARNING – this feature has not been fully tested and should be used with caution. Please contact Trilinear Technologies technical support before enabling this feature. 4:0 – Number of DSC slices to process on each hardware slice processor. The total number of slices is $HARDWARE_SLICE_COUNT * SLICES_PER_PROCESSOR$.
0x04C	RW	SLICE_PROCESSOR_COUNT Enables the specified number of hardware slice processors. In most operational cases, this value should be set to the total number of implemented hardware processors. Lower power encoding can be realized by setting this register to the minimum number of hardware processors needed to encode the input video mode. 3:0 – Set to the number of enabled slice processors minus 1.
Interrupt Control and Status		
0x080	RW	INTERRUPT_ENABLE This register controls which core interrupts are enabled. Enabled interrupts will result in the assertion of the host interrupt output signal. All bits in this register default to 0 at power up or a core reset. A soft reset does not affect the contents of this register. 5 – IE_TIMER: Set to a 1 to allow the internal timer to generate an interrupt. 4- IE_FRAME_COUNT: Set to a 1 to enable the frame count interrupt. An interrupt will be generated after the number of frames specified by the FRAME_INTERRUPT_COUNT register have been encoded. 3 – IE_RATE_ERROR: Set to a 1 to enable stream errors to cause interrupts. 2 – IE_END_OF_FRAME: Set to a 1 to enable an interrupt to be asserted at the end of every encoded frame. The interrupt is generated when the last data has been read from the AXI output interface. 1 – IE_END_OF_SLICE: Set to a 1 to enable an interrupt to be asserted at the end of every encoded slice. This interrupt source only applies to slice processor 0. 0 – IE_ENCODER_TIMEOUT: Set to a 1 to enable the parser timeout interrupt. A timeout will only occur under certain error conditions in the picture parameter set that will cause the encoding process to stall. When the picture parameter set is properly configured, this interrupt should never be asserted.
0x084	RO	INTERRUPT_CAUSE Determines which event in the core has caused the core interrupt signal to be asserted. When an interrupt is asserted, one or more bits of this register will be set to 1. Reading from this register will cause all interrupts to be cleared. 5 – IC_TIMER: Set to a 1 when the internal timer block has caused an interrupt. 4 – IC_FRAME_COUNT: Asserts when a number of frames equal to the FRAME_INTERRUPT_COUNT register have been decoded.



		3 – IC_RATE_ERROR: Asserts when the internal parsing logic has detected an overflow of the internal rate buffer. 2 – IC_END_OF_FRAME: Asserts when the parsing engine has completed the encode of a single frame. 1 – IC_END_OF_SLICE: Asserts when the parsing logic has completed encoding the current slice. 0 – IC_ENCODER_TIMEOUT: The timeout interrupt asserts when the internal parsing timer has expired. This timer expires after $\text{PARSER_TIMEOUT_COUNT} \times 1024 \text{ AXI_CLK}$ cycles have passed without activity from the compression engine.
0x088	RO	INTERRUPT_STATE This register contains the unfiltered state of the event detection bits. When an interrupt event is detected, this bit is always set independently of the state of the INTERRUPT_ENABLE register. The state bits in this register may be used for a polling based event detection system. 5 – IS_TIMER: Set the timer interrupt has triggered. 4 – IS_FRAME_COUNT: Set when the frame count interrupt condition is detected. 3 – IS_RATE_ERROR: When a rate error has been detected, this bit will be set to a 1. 2 – IS_END_OF_FRAME: When the end of the source frame has been detected, this bit will be set to a 1. 1 – IS_END_OF_SLICE: When the end of the current slice is detected, this interrupt bit will be set to a 1. 0 – IS_PARSER_TIMEOUT: The timeout interrupt asserts when the internal parsing timer has expired. This timer expires after $\text{PARSER_TIMEOUT_COUNT} \times 1024 \text{ AVC_CLK}$ cycles have passed without activity from the parsing engine.
0x08C	RW	FRAME_INTERRUPT_COUNT Allows the internal controller to assert an interrupt to the host processor after the specified number of frames have been processed by the encoder core. A frame is completely encoded after all slices in a frame have been output from the rate buffer. 7:0 – The number of frames to encode before asserting an interrupt to the host processor. A value of 0 is invalid and will never assert an interrupt.
Core Revision		
0x0F8	RO	CORE_FEATURES Reports the features present in a specific implementation of the DSC encoder core. This register may be used by software to determine the configuration of the core. 31:16 – MAX_LINE_SIZE: Indicates the maximum number of pixels per line that may be encoded by the compression engine. 8 – BLOCK_PREDICTION_PRESENT: Set to a 1 when the implemented core includes the block prediction feature. 3:0 – SLICE_PROCESSOR_COUNT: Indicates the number of slice processors present.
0x0FC	RO	CORE_ID Returns the unique identification code of the core and the current revision level. 31:16 – Core ID fixed at 0x0031. 15:0 – Core revision level in the form Major [15:8], Minor [7:0].
Picture Parameter Set		
0x100	RW	PPS_TABLE_DATA



		Picture Parameter Set update area for the access of the PPS by the host processor. The least significant 8 bits of this register are used for the PPS table entry. Bits 31:8 of this address offset is unused. 7:0 – A write stores the data value at the current entry selected by the internal data pointer.
0x104	RW	PPS_TABLE_ENTRY Sets the current picture parameter set table entry for access through the PPS_TABLE_DATA register. The table entry value will automatically increment after each read or write access. 6:0 – Index of the next PPS table data access.
0x108	RW	PPS_TABLE_COMMIT This control register is used to determine when the PPS_UPDATE_TABLE is transferred to the active parameter set. 0 - Writing a 1 to this bit updates the current Picture Parameter Set in use by the decoder at the next frame boundary.

Table 5-1 : Register Table